

2주차 2차시 [통신의 오류 제어]

강의 주제 1 통신에서 오류의 발생

1. 오류 발생 원인

1) 오류 제어 개요

- 우리가 원하는 장소에 전송매체를 통해 데이터 전송을 하게 되면 전송로에 따른 잡음이나 장비의 고장 등 여러 가지 요소들 의하여 필연적으로 데이터에 오류가 발생한다.
- 오류란 보낸 데이터와 받는 데이터 사이에 차이가 있는 경우를 말하는데,
- 즉, 보낸 정보가 '1'을 전송했을 때 '0'을 수신하게 되면 오류
- 통신에 있어서 송신측의 올바른 정보를 받기 위해서 수신측에서 이를 해결하는 과정이 필요하다.
- 이를 해결하는 과정을 오류제어(Error Control)라 한다.

2) 통신에서 오류의 종류

- 위성 통신회선 등에서 볼 수 있는 랜덤 오류(Random Error)
- 유선상에서 임펄스성 잡음, 무선 환경에서 페이딩에 의한 버스트 오류
- 각종 기기의 오동작, 전송로상의 잡음, 인위적 오류
 - 버스트 오류는 단일 비트 오류보다 일어날 가능성이 높고, 전송률, 잡음 지속시간에 영향을 받는다.

3) 오류의 원인과 해결

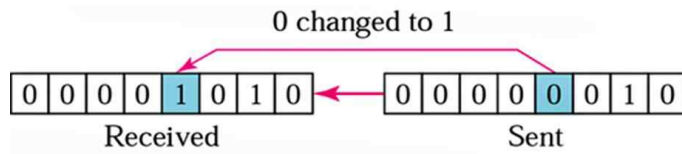
- 오류 허용 오류율: $10^{-5} \sim 10^{-9}$ CCITT 기준
- 오류는 어느 정도 혼합된 상태로 발생하게 되고, 통신 과정에서 송신측의 올바른 정보를 수신하기 위해서는 수신측에서 이를 해결하는 과정 필요
- Channel Coding : 전송 채널의 결함(noise, fading, etc)에 의한 품질(BER) 저하를 극복

2. 데이터 오류의 형태

- 수신 측으로 전송한 데이터는 송신 측의 데이터와 동일해야 하지만, 다양한 원인 때문에 데이터 오류가 발생할 수 있다.
- 오류 형태는 단일비트오류, 다중 비트 오류, 집단 오류로 분류할 수 있다.

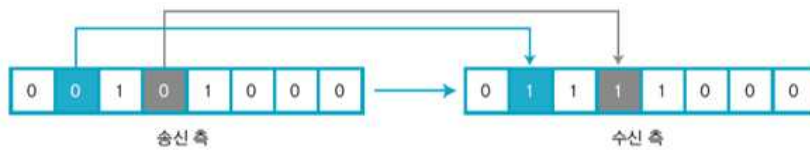
1) 단일 비트 오류(Single-bit Error)

- 주어진 데이터 단위(한 바이트의 문자)중 하나의 비트만 변경되는 오류



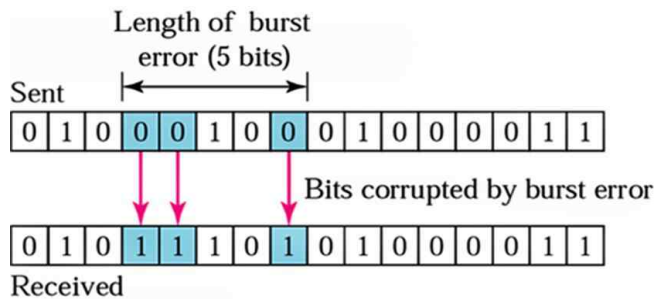
2) 다중 비트 오류(Multiple-bit Error)

- 데이터 단위 중 2개 이상의 비연속적인 비트가 변경되는 오류



3) 집단 오류(Burst Error)

- 데이터 단위 중 2개 또는 그 이상의 연속적인 비트가 변경되는 오류.
- 송신 측이 보내려는 데이터 외에 별도로 잉여 데이터를 추가해서 전송하면 수신 측은 이 잉여 데이터를 검사하여 오류를 검출할 수 있다.



강의 주제 2 통신 오류 검출 기법

1. 오류의 검출과 정정 방법

1) 순방향 오류 제어 (FEC, Forward Error Control)

- 순방향 오류 정정 방식은 연속적인 데이터 흐름이 필요하거나, 역방향 채널의 제공이 어려울 때 많이 사용한다.
- 순방향 오류 정정 방식은 수신측에서 오류를 검출할 뿐 만 아니라 수정할 수 있는 방식으로 더 많은 패리티 비트를 필요로 하게 된다.
- 오류 정정을 위해 부가 비트를 추가하므로 전송 효율이 저하되는 단점이 있다.
- 위성통신, 1대 n 방식의 통신에 이용되며, 순방향 오류 정정의 대표적인 부호로 해밍 (Hamming) 부호, 콘볼루션 코드 등이 있다.

2) 후진 오류 제어(BEC, Backward Error Correction)

- 대부분의 데이터 통신에서 이 방식을 택하고 있다.

- 송신측에서 에러 검출이 가능한 부가적 정보(Redundancy)를 메시지에 첨가시켜 전송하고,
- 수신측이 이를 토대로 에러를 발견시 재전송을 송신측에 요구하는 ARQ(자동반복 요청 (Automatic Repeat reQuest) 방식이다.
- ARQ 방식은 역 방향 채널을 이용하는 오류 검출 방식으로 ARQ 종류는
 - Stop and Wait: 한 번에 하나씩 긍정 확인응답(ACK)을 받고 데이터 전송
 - Go Back N: 오류 발생 프레임 이후 모두 재전송 요청하는 방식
 - Selective Repeat: 오류가 발생한 프레임만 재전송 요청하는 방식
 - Adaptive ARQ : 적응적으로 ARQ 횟수를 줄여 전송 효율을 높이는 방식

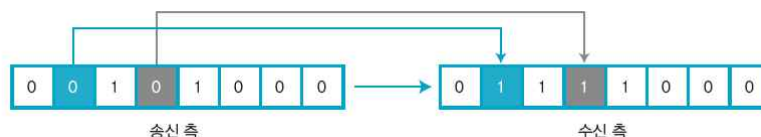
2. 통신 오류 검출 방식

- 수신 측으로 전송한 데이터는 송신 측의 데이터와 동일해야 하지만, 다양한 원인 때문에 데이터 오류가 발생할 수 있다.
- 따라서 신뢰할 수 있는 네트워크 통신을 하기 위해서 오류를 검출 및 수정을 해야 한다.
- 네트워크 통신에서 많이 사용되는 오류 검출 방법은 다음과 같다.
 - 패리티 검사(Parity check)
 - 블록합 검사(Block Sum Check)
 - 순환 중복 검사(CRC: Cyclic redundancy Check)
 - 검사합(Checksum Check)



1) 패리티 검사(Parity Bit Check)방식

- 패리티 비트 검사는 전송하는 데이터마다 패리티 비트(parity bit)라는 중복 비트를 데이터 단위에 덧붙인다.
- 패리티 비트를 포함한 데이터 단위 내 1의 전체 개수가 짝수(또는 홀수)가 되도록 해야 함
- 송신 호스트와 수신 호스트는 동일한 패리티 방식을 사용해야 한다.
 - 짝수 패리티(even parity) : 1의 전체 개수가 짝수가 되도록 함

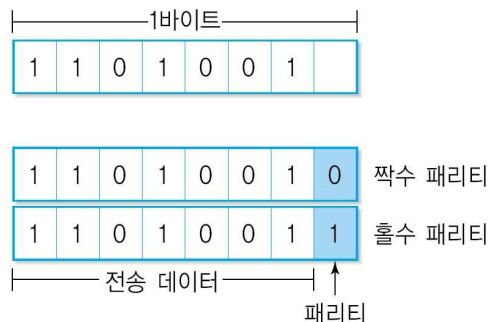


- 홀수 패리티(odd parity) : 1의 전체 개수가 홀수가 되도록 함



(1) 패리티 검사 방식의 인코딩(Encoding)

- 7비트 데이터가 1101001인 경우를 살펴보자.
- 1 바이트 = 7 비트 ASCII 코드 + 1 비트 패리티
- 짝수 패리티 방식을 사용하면 1101001에서 1이 4개이므로 짝수이다.
- 여기에 패리티 비트는 0을 추가하면 전송되는 전체 데이터에 1의 개수는 짝수가 된다
-> 11010010
- 송신 호스트와 수신 호스트는 동일한 패리티 방식을 사용해야 함



(2) 패리티 검사 방식의 디코딩(Decoding)

① 짝수 패리티 검사 방식을 사용하면

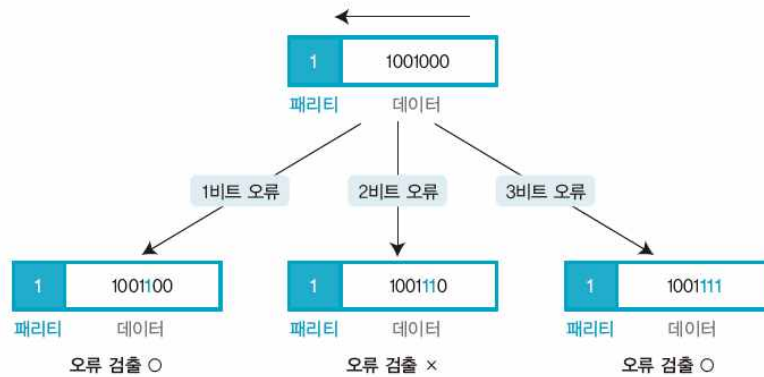
- 11010010 에서 패리티 비트는 0이므로 1의 개수가 짝수인지 확인한다.
- 패리티 검사는 패리티 비트를 포함해서 각각 XOR 연산을 한 후 결과,
-> 0(1의 개수가 짝수)이면 오류가 없는 것
-> 1(1의 개수가 홀수)이면 오류가 검출된 것
- 검사 : $1 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 = 0$

② 홀수 패리티 검사 방식을 사용하면

- 전송하려는 데이터가 1101001이라고 가정해 보자.
- 1의 개수를 홀수로 만들려고 패리티 비트를 1로 지정하면 11010011
- 데이터를 전송 받은 수신 측은 패리티 비트를 포함한 데이터 1의 개수를 계산하여 홀수인지 판단한 후 홀수가 아니면 재전송을 요청한다.
-> 1(1의 개수가 홀수)이면 오류가 없는 것
-> 0(1의 개수가 짝수)이면 오류가 검출된 것
- 검사 : $1 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 1 = 1$

(3) 패리티 검사 방식의 단점

- 패리티 비트를 이용한 오류 검출 방식은 단일 비트의 오류는 검출할 수 있으나 짝수개의 오류가 발생하면 에러를 검출할 수 없는 단점이 있다.



(4) 패리티 방식 검사 예



송신자가 'world'라는 단어를 보내고자 합니다.

① ASCII 코드를 사용하면 다섯 글자는 다음처럼 코드화된다.

1110111 1101111 1110010 1101100 1100100

② 실제 전송되는 비트는 다음과 같다.

11101110 11011110 11100100 11011000 11001001

③ 'world'라는 단어가 전송시 변환되지 않고 수신자에게 수신되었다면,

11101110 11011110 11100100 11011000 11001001

④ 수신자는 각 글자에서 1의 수를 세고 짝수 (6, 6, 4, 4, 4)임을 알고서 그 단어는 받아들여진다.

⑤ 'world'라는 단어가 전송 중 오류되었고, 이를 수신자가 수신했다면,

11111110 11011110 11101100 11011000 11001001

⑥ 수신자는 각 글자에서 1의 수를 세고 짝수, 홀수(7, 6, 5, 4, 4)이므로, 데이터가 변환되었음을 알게 되어, 단어를 버리고 재전송을 요청한다.

2) 블록 합 검사(Block Sum Check)

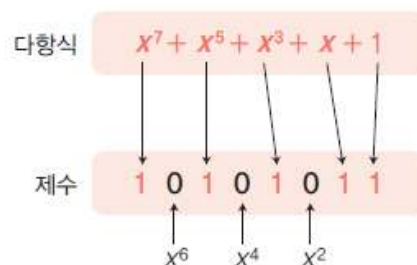
- 패리티 비트를 이용한 방식은 오류가 발생한 비트의 개수가 짝수일 때 오류를 검출할 수 없으므로 이러한 문제를 해결하기 위한 방법이다.
- 오류 검출 능력을 향상시키려고 문자 블록에 수평 패리티 검사와 수직 패리티 검사를 2차원적으로 검사하는 방법.
- 행 단위 패리티에 열 단위 오류 검사를 수행할 수 있는 열 단위 패리티를 추가하여 이중으로 오류 검출 작업을 수행한다.
- 추가된 열 패리티 문자: 블록 검사 문자(BCC, Block Check Character)
- 단점 : 검사데이터를 추가로 전송하기 때문에 패리티 방식보다 더 많은 오버헤드가 발생한다.

데이터	데이터 비트	패리티 비트
데이터 1	0001011	1
데이터 2	1100111	1
데이터 3	0000011	0
데이터 4	1010001	1
데이터 5	1111101	0
블록 검사 문자	1000011	1

3) 순환 중복 검사(CRC, Cyclic Redundancy Check)

- (1) 정확하게 오류를 검출하려고 다항식 코드를 사용하는 방법이다.
- (2) 오류가 없을 때는 계속 발생하지 않다가 오류가 발생하면 그 주위에 집중적으로 오류를 발생시키는 집단 오류를 검출하는 능력이 탁월하고,
- (3) 하드웨어나 소프트웨어로 쉽게 구현이 가능하고, 동작이 빠르다.
- (4) 생성다항식(Polynomial) : CRC발생기는 0과 1의 스트링 보다는 대수 다항식으로 표현한다.

(예)



(5) 다항식을 이용한 순환 중복 검사의 오류 검출 과정

- ① 송신측이 전송하기 전에 송수신측은 동일한 생성 다항식을 결정
- ② 송신측에서 K비트의 전송 데이터를 생성 다항식으로 나눈 나머지 값 n비트를 구한다.
- ③ K비트의 전송 데이터에 나머지 값인 n비트 값을 추가하여 K+n비트 데이터를 전송한다.
- ④ 수신 측에서는 수신된 K+n비트의 데이터를 생성 다항식으로 나눈다.
- ⑤ 나눈 나머지가 0이면 오류가 없는 것, 0이 아니면 오류가 발생한 것이다.

(6) CRC가 채택하고 있는 표준 다항식

(3) 검사합의 검사기 동작 과정

- ① 수신기는 데이터 단위를 나눈다.
- ② 모든 세그먼트들을 더하고 결과를 보수화 한다.
- ③ 데이터 세그먼트와 검사합을 더한 전체 값이 0이면 데이터가 손상되지 않았다는 의미
- ④ 0이 아니라면 수신기는 이를 거부한다.

(4) 검사합 방식 예1

16비트 블록을 8비트 검사합을 사용하여 보낸다고 가정하자.

10101001 00111001

- ① 16비트 블록으로 8비트 검사합을 생성하고 원래데이터와 검사합 데이터를 송신한다.

	10101001
	00111001

Sum	11100010
Checksum	00011101
	10101001 00111001 00011101

- ② 수신자는 송신자가 보낸 비트열을 받았고 오류는 없다고 가정하자.

10101001 00111001 00011101

- ③ 수신자가 3개의 섹션 모두를 더한 후, 모두 1의 결과를 얻을 것이다.
보수화 하면 모두 0이고 오류가 없음을 보여준다.

	10101001
	00111001
	00011101

Sum	11111111
Complement	00000000

5) 검사합 방식 예2

16비트 블록을 8비트 검사합을 사용하여 보낸다고 가정하자.

- ① 이제 4비트에 영향을 주는 길이 5의 폭주 오류가 있다고 가정하자.

10101111 11111001 00011101

- ② 수신자가 3개의 섹션을 모두 더하면 다음과 같이 오류를 알게 된다.

	10101111
	11111001
	00011101

Result 1	11000101
Carry	1
Sum	11000110
Complement	00111001 (이 비트 형식은 변환되었음을 의미)